Drevet Loïc

Hopital Max

Gault Mathias

R3.04 – LE PATTERN VISITEUR



BUT2

Année 2026 – 2027

Sommaire:

1. L’objectif du pattern visiteur - page 3

2. Rapport de Visiteur avec SOLID - page3 - 4

3. Limites et Forces du pattern -
   page 4

4. Détails des classes de Visiteur - page 5

5. Exemple et problématique - page 5

1) L’objectif du pattern Visiteur :

Le but des design pattern est de mettre en place une solution simple à un problème connu. Il y a 23 design pattern séparés en 3 catégories.

- Créationnels
- Structurels
- Comportementaux

Le pattern Visiteur est un pattern comportemental

Une fois une classe créée on peut ne pas vouloir la changer mais aussi vouloir ajouter de nouvelles fonctionnalités.

Le but du pattern visiteur est de permettre l’ajout de nouvelles méthodes dans une classe à travers une autre classe. En ajoutant une interface Visiteur nous pouvons créer une méthode “accepter” pour les objets que l’on veut pouvoir modifier prenant un élément de Visiteur comme entrée. Chaque élément implémentant Visiteur doit avoir une méthode pour chaque classe avec “accepter”.

2) Rapport de Visiteur avec SOLID :

- SRP L’ajout de nouvelle méthode se fait dans des classes séparées. Chaque classe a sa propre responsabilité.
- OCP L’objectif de Visiteur est d’ajouter des méthodes sans modifier les classes existantes, en revanche il faut les modifier classes pour que Visiteur puisse fonctionner. Il faut aussi modifier les classes visiteurs à chaque ajout d’un nouvel objet.
- LSP Tout objet de la sous class de l’interface doit pouvoir utiliser l’interface Visiteur.

- ISP Visiteur ne respecte pas puis ce qu'il fait appelle à une interface qui fonctionne pour un ensemble d'élément donnée.
- DIP Le pattern visiteur fait que les classes dépendent d'une interface et non d'une abstraction. Les dépendances sont séparées

3-1) Limites :

L'ajout de matériel est la faiblesse principale du pattern visiteur car pour chaque objet qu'on pourrait vouloir ajouter on doit modifier l'interface ainsi que les classes ConcreteVisitor à chaque fois qu'on veut rajouter un élément.

3-2) Forces :

L'ajout d'opérations est la force principale du pattern Visiteur car il permet de rajouter des méthodes pour des classes "ElementConcrete" sans avoir à les modifier grâce à l'intermédiaire de visiteurs. Cela permet d'avoir une vision plus claire de son projet et d'éviter les classes couteaux-suisses.

4)Détail des classes de Visiteur :

Nom de classe	Role
Element	Interface qui donne le accept() aux éléments et permet donc la visite
ConcreteElement	Objets de la classe qui sont visités
Visitor	Interface qui donne le visit() aux classes méthodes et permet donc de les effectuer sur les éléments concrets
ConcreteVisitor	Classes visiteurs ayant chacune un rôle de méthode à effectuer

5) Exemple et problématique

Le premier exemple est celui d'un parc de matériel informatique. Une interface Materiel (allumer, éteindre) est implémentée par trois classes : Imprimante, Serveur et BoxWifi. Chacune a ses propres attributs (adresse IP, nombre de feuilles, capacité…) et ses opérations (imprimer, scanner, redémarrer le service, calibrer le réseau…).

Dans cette solution sans visiteur, toutes les opérations sont écrites dans les classes du matériel. Dès qu'on veut ajouter une nouvelle fonction, par exemple mesurer la consommation ou vérifier l'état de sécurité, il faut modifier chaque classe de la hiérarchie. Ces classes accumulent aussi des responsabilités qui ne sont pas vraiment les leurs.

6) Bibliographie

- bouhours.net
- alexsoyes.com
- ionos.fr